

Sahayi — Hackathon Project Spec

Sep 25, 2026 · @Ejin

Overview

Sahayi ("helper" in Malayalam) is a voice-first crypto wallet for blind users. You speak in your own language mix, there is no seed phrase to lose, and every change on screen is announced.

The problem, backed by research:

- A 2023 study tested MetaMask with 23 blind users. Blind users took about 70% longer than sighted users (47.9 vs. 28.2 minutes) to finish basic tasks.
- Actions like copying an address or submitting a transaction gave no screen reader announcement. Even the researchers' improved wallet still had confirmation popups that blind users missed.
- Managing a 12-word seed phrase with a screen reader was so tiring that some blind users skipped the backup. One could not recover her wallet later.
- Blind users cannot easily tell long wallet addresses apart, and several sent funds to the wrong address.
- The authors name the next step themselves: security that removes the need for a seed phrase.

A separate 2026 review of banking apps for visually impaired users found authentication barriers in 18 studies. When apps aren't accessible, users hand their phone and PIN to someone else, which increases fraud risk.

One-line pitch: "Researchers showed crypto wallets fail blind users and said the fix is removing seed phrases and fixing notifications. We built it, in the user's own language."

Scope: testnet only, with fake funds. No real bank or real money.

How Sahayi covers the three topics

Each topic solves one real failure for the same user. None is decoration.

Topic	Failure it fixes	What we build
AI/ML: code-switching and spelling by ear	Voice assistants expect clean English; blind users can't handle long addresses	Voice commands in Manglish, Hinglish, Arabic-English, or English, turned into actions like "send 500 to Amma"
Blockchain: self-custody assumes you are alive	Seed phrases are inaccessible, and lost keys or death freeze funds forever	A seedless smart wallet with guardian recovery and an inheritance switch
Web Dev: interfaces that change silently	Balance updates, confirmations, and errors appear without being announced	Every state change announced, with focus management and distinct sounds

Demo flow

The demo runs about 4 minutes with the screen reader on and the screen dimmed, so judges experience it by sound.

1. **Setup without a seed phrase.** The user creates a wallet and adds two guardians by contact name. No 12 words appear. Each step is announced.
2. **Check balance by voice.** The user says "ente balance ethra?" The app answers "Your balance is 2.5 test ETH" and announces it.
3. **Send money by voice.** The user says "Amma-kku 0.1 ayakku." The app reads back "Send 0.1 ETH to Amma. Confirm with fingerprint." After approval, it announces "Pending," then "Confirmed," each with a distinct sound.
4. **A judge tries it.** A judge speaks a command in their own language mix.
5. **Lost phone.** On a second device, the user starts recovery. Two guardians approve, a short delay passes, and access returns. The original owner could cancel during the delay.
6. **Inheritance.** With a 2-minute demo timer, the owner goes inactive and the beneficiary claims the funds.
7. **Before/after.** Play a clip of the same steps in a normal wallet: silence where Sahayi speaks.

Backup: record a full demo video by hour 31.

Architecture and tech stack

Four parts: a web app, a speech-to-text step, an intent API, and a smart contract on testnet. The AI never moves money; it only proposes an action that the user confirms with a fingerprint.

flowchart LR

```

    U[User speaks] --> S[Speech to text]
    S --> I[Intent API<br/>code-mixed NLU]
    I -->|proposed action| W[Web app<br/>reads it back]
    W -->|fingerprint approve| K[Device key]
    K -->|signed tx| C[SahayiVault<br/>contract]
    C -->|events| W
    W --> A[Announcer<br/>live region + sounds]
  
```

Part	Tech
Web app	React + Vite, installable as a PWA on Android
Speech to text	Browser Web Speech API first; Whisper or AI4Bharat IndicConformer as upgrade
Intent API	Python + FastAPI; fine-tuned MuRIL or Laya for intent, rules for amounts
Approval	WebAuthn passkey (the phone's fingerprint or face unlock)
Wallet key	Key generated on the device, unlocked only after passkey approval
Contract	Solidity, tested with Foundry or Hardhat, deployed on a public testnet
Chain access	ethers.js or viem

Typed fallback: every voice action also works by typing, in case speech recognition fails on stage.

AI/ML module: voice commands

The model turns a messy, code-mixed sentence into one structured action with a confidence score. It must handle all three language challenges from the brief.

Intents (keep to 6):

Intent	Example (Manglish / Hinglish)
check_balance	"ente balance ethra?" / "mera balance kitna hai"
send	"Amma-kku 500 ayakku" / "Rahul ko 500 bhejo"
history	"last transaction enthaayirunnu?"
add_contact	"Rahul-ne contact aayi add cheyyu"
recovery_help	"phone poyi, enthu cheyyanam?"
cancel	"venda" / "rehne do"

Challenges it must handle:

- **Code-switching:** English words like "balance" inside Malayalam or Hindi sentences.
- **Spelling by ear:** speech-to-text returns inconsistent spellings (ayakku, ayakk, ayaku). Normalize with a variant dictionary plus phonetic matching.
- **Transliteration:** output may arrive in Malayalam script or Latin letters. Convert both to one form before classifying.
- **Spoken numbers:** "anjooru" (500), "aayiram" (1000), "paanch sau" (500). Use a rules-based number parser; don't rely on the model.
- **Contact matching:** "Amma", "amma", and "അമ്മ" all match the saved contact, using phonetic matching.

API: POST /intent with { "text": "...", "contacts": ["Amma", "Rahul"] } returns:

```
{ "intent": "send", "amount": 500, "contact": "Amma", "confidence": 0.93,
  "lang_mix": ["ml", "en"] }
```

Confidence rule: below 0.8, the app asks a clarifying question instead of acting, like "Did you mean send or check balance?"

Data: the team writes about 50 commands per intent per language, then an LLM expands them to about 3,000 with spelling variations. Hold back 150 hand-written commands as the test set.

Model: fine-tune MuRIL for intent classification (about 1 hour on a free Colab GPU). Laya is an option to test, but its base model needs fine-tuning, so keep MuRIL as the default.

Numbers for the pitch: intent accuracy and amount-extraction accuracy on the test set, compared against a plain English-only baseline.

Blockchain module: SahayiVault contract

One contract holds the funds. The owner's device key controls it day to day. Guardians can restore access if the device is lost, and a beneficiary inherits if the owner is gone. No one ever sees a seed phrase.

Settings (set at creation): owner key, guardians (2 or 3), approval threshold (for example, 2 of 3), beneficiary, inactivity period, recovery delay, and inheritance grace period. For the demo, all timers are 2 minutes.

Owner functions:

- `send(to, amount)` : transfers funds and resets the activity timer.
- `ping()` : resets the activity timer without sending.
- `cancelRecovery()` : stops a recovery the owner didn't ask for.

Social recovery (lost phone):

1. A guardian calls `proposeRecovery(newOwner)` .
2. Other guardians call `approveRecovery(newOwner)` .
3. When the threshold is reached, a delay starts.
4. After the delay, anyone calls `executeRecovery()` and the new device becomes the owner.

If the real owner acts during the delay, the recovery is cancelled. That stops guardians from teaming up to steal access.

Inheritance switch (owner gone):

1. After the inactivity period passes with no owner activity, a grace period starts.
2. During the grace period, any guardian can call `vetoInheritance()` if they know the owner is alive. This resets the timer.
3. After the grace period, the beneficiary calls `claimInheritance()` and receives the funds.

Rules:

- Guardians can never move funds themselves. They can only approve a new owner key.
- Only one recovery can be active at a time.

- Every state change emits an event (`Sent` , `RecoveryProposed` , `RecoveryApproved` , `RecoveryExecuted` , `RecoveryCancelled` , `InheritanceStarted` , `InheritanceVetoed` , `InheritanceClaimed`). The web app listens to these events and announces each one.

Contacts: names like "Amma" map to addresses in an address book stored on the device, not on-chain. Users never hear or type an address.

Tests to write: recovery with threshold, recovery cancelled by owner, inheritance vetoed, inheritance claimed, and guardians unable to send funds.

Web module: accessible UI

Nothing changes on screen without being announced. This directly fixes the missed confirmations found in the research.

Announcer:

- One `aria-live="polite"` region and one `role="alert"` region, both created when the app loads.
- Polite for normal updates (balance loaded, transaction confirmed). Alert only for errors and security events (recovery started on your wallet).
- Clear the region, wait about 100 ms, then insert new text, so repeated messages are still announced.
- Every contract event is mapped to a spoken message.

Distinct sounds (short, under 1 second): listening started, success, error, and security warning. Sounds add to announcements; they never replace them.

Focus management:

- Confirmation steps use a real `<dialog>`. Focus moves into it and returns to the previous control when it closes. The research found popups that screen readers didn't notice; a proper dialog fixes that.
- Focus never jumps unexpectedly during typing or voice input.

Other requirements:

- Every button has a clear, descriptive label, like "Send money" rather than "Next."
- Explain terms in plain words next to them, like "Wallet address: like a bank account number."
- No timeouts on any step. The user can take as long as needed.
- Money amounts are read in full words and currency, like "zero point one test ETH."

- Full keyboard use, plus one large on-screen button to start voice input.

Testing: check every flow with NVDA on Windows and TalkBack on Android. Add a Playwright test that confirms each contract event produces an announcement.

Security decisions

Judges will ask about security, so these choices are deliberate and should be stated in the pitch.

Decision	Why
No voice password; fingerprint or face unlock instead	Voice biometrics are being abandoned: a clone can be made from about 3 seconds of audio, and 91% of US banks surveyed are rethinking voice authentication. Speaking a password aloud also lets anyone nearby hear it.
The AI never moves money	It only proposes an action. The app reads it back, and the user approves with a fingerprint. A misheard command can't send funds.
No personal or banking data on the blockchain	Public blockchains are visible to everyone and permanent. The chain only holds funds and recovery rules.
No seed phrase	The research showed seed phrases are the biggest barrier for blind users. Guardian recovery replaces them.
Recovery has a delay the owner can cancel	Stops guardians from taking over the wallet without the owner knowing.
Guardians can't send funds	Their only power is approving a new owner key.
Voice commands processed on our own server, not a third-party LLM	Financial commands are private.

Say this honestly: it's a testnet prototype and has not been security-audited.

Team split and 36-hour timeline

The full demo must work end to end by hour 18. After that, only polish, numbers, and rehearsal.

Hours	ML (you)	Frontend	Blockchain
0-3	Stub <code>/intent</code> returning fixed output; test speech-to-text on Manglish	App skeleton, NVDA and TalkBack test	Contract skeleton, testnet setup
3-10	Write commands, LLM expansion, number parser	Voice button, announcer, read-back dialog	Recovery and inheritance logic plus tests
10-16	Fine-tune MuRIL, real <code>/intent</code>	Passkey approval, contacts, balance screen	Deploy to testnet, event list for frontend
16-18	Integrate	Integrate	Integrate
18-20	Fix misread commands from testing	Sounds, focus fixes	Guardian and beneficiary screens
20-25	Sleep in shifts	Sleep in shifts	Sleep in shifts
25-31	Accuracy numbers on test set	Before/after clip, Playwright test	Demo timers, edge cases
31-34	Pitch slides	Record backup video	Rehearse demo
34-36	Buffer	Buffer	Buffer

If the team has only 2 people, the frontend person also takes the contract. Keep it to recovery first, then inheritance.

Cut in this order if behind: Hinglish and Arabic support, then inheritance, then the before/after clip. Never cut the read-back and fingerprint confirmation.

Done means:

- ☐ Balance check and a send work fully by voice in Manglish
- ☐ Every send is read back and approved with a fingerprint
- ☐ Guardian recovery works on a second device, and the owner can cancel it

- ☐ Inheritance claim works with the 2-minute demo timer
- ☐ Every contract event is announced, verified with a screen reader
- ☐ Intent accuracy measured on 150 held-out commands
- ☐ Backup demo video recorded

Sources

- [Iterative Design of An Accessible Crypto Wallet for Blind Users](#) (Zhou et al., UIUC and CMU): MetaMask study with 23 blind users, iWallet redesign, and future work on removing seed phrases
- [Accessibility of Digital Financial Applications for People With Visual Impairment: Scoping Review](#) (JMIR, 2026): authentication barriers across studies
- [Toward Accessible Mobile Money: A Voice-Driven USSD Framework for Visually Impaired Users](#): users sharing PINs with helpers and the resulting fraud risk
- [Voice vs Device Biometrics: The Security Shift in Banking](#) (interface.ai): BioCatch survey on banks rethinking voice authentication
- [Passive Biometrics Solution for Banks](#) (Matellio): voice clones from about 3 seconds of audio
- [DravidianCodeMix dataset](#): Malayalam-English code-mixed text for training